

Stack Applications: Well-Formedness of Parentheses

Q. Explain how a stack can be used to check the well-formedness of parentheses in an expression. Write a step-by-step algorithmic approach.

****Definition to Remember****

An expression is well-formed if every opening bracket has a corresponding closing bracket.

1. Why Stack Works for This Problem

* LIFO ensures the **most recently opened bracket** is at the top of the stack.

* When

2. Algorithm

1. Initialize an empty stack.

2. Scan

3. Trace Example: `{ [ ( ) ] }`

Character	Action	Stack State
-----------	--------	-------------

{	Push	{
[	Push	{, [
(	Push	{, [, (
)	Pop	{, [,]
]	Pop	{, }
}	Pop	

Invalid Example: `{ ( ) }`

* Reac

****Must-Write Points (for 10 marks)****

1. Stack

Quick Recall

`Opening
Non-em

C Code: Parentheses Checking Algorithm using Stack

```
#include <stdio.h>
```

```
#include
```

```
#define MAX 100
```

```
struct CharStack {
```

```
char ite
```

```
void push(struct CharStack *s, char c) {
```

```
s->item
```

```
char pop(struct CharStack *s) {
```

```
if (s->to
```

```
int isMatchingPair(char char1, char char2) {
```

```
if (char2
```

```
int checkBalanced(char exp[]) {
```

```
struct C
```

```
for (int i = 0; i < strlen(exp); i++) {
```

```
if (exp[i]
```

```
int main() {
```

```
char ex
```

```
printf("Expression: %s -> %s\n", expr1, checkBalanced(expr1) ? "Balanced " : "Unbalanced ");
```

```
printf("E
```

```
return 0;
```

```
}
```